
Guía de Ejercicios N°4 – Ciclos en C

Ayudante: Matías Fuentes

1. Detector de Drones

Un sistema de vigilancia recibe señales enteras que representan lecturas captadas desde el cielo. Cada señal puede ser **1** (se detectó movimiento) o **0** (no se detectó movimiento). El sistema debe leer señales de forma continua hasta que se ingrese el valor **-1**, el cual indica el término de la transmisión y no debe considerarse una señal válida.

Cada vez que el sistema reciba una nueva señal, debe evaluar las **últimas 3 señales captadas** (la actual y las dos anteriores): si las tres señales consecutivas son iguales a 1, el sistema debe imprimir "¡Alerta! posible dron detectado". Si aún no se han recibido al menos 3 señales, no se debe realizar ninguna evaluación.

Hint: Puedes resolverlo utilizando tres variables enteras que vayan "desplazando" los valores recibidos (por ejemplo, antes de leer la nueva señal, actualiza: la más antigua toma el valor de la intermedia, la intermedia toma el valor de la más reciente, y luego lees el nuevo valor). Recuerda que **no debes utilizar** `while(1)` para tu ciclo principal: apóyate en la condición de lectura dentro del `while` o `do-while`.

2. Número Palíndromo

Escriba un programa en C que solicite números enteros positivos de un solo dígito (entre 0 y 9) al usuario, uno a la vez, e ir concatenándolos para formar un único número. El programa debe seguir solicitando dígitos hasta que el usuario ingrese un valor **fuera del rango** 0-9, momento en el cual se detiene la lectura (ese último valor no se concatena).

Una vez formado el número completo, el programa debe:

- Calcular su número **invertido** (el número "dado vuelta").
- Indicar si el número original es **palíndromo** (es decir, si es igual a su versión invertida).
- Indicar cuál de los dos números es **mayor**, en caso de que no sean iguales.

Hint: Para ir concatenando dígitos puedes usar la operación `numero = numero * 10 + digito`. Para invertir un número entero, utiliza un ciclo que vaya extrayendo el último dígito con el operador módulo (`% 10`) y lo vaya agregando a un nuevo número, mientras reduces el original con división entera (`/ 10`).

3. Contador de Aterrizajes

Una pista de aterrizaje recibe señales enteras que representan la velocidad (en km/h) de las naves que se acercan. El sistema debe leer velocidades de forma continua hasta que se ingrese un valor **negativo**, el cual indica el cierre de la pista y no debe considerarse una lectura válida.

El programa debe contar cuántas naves aterrizaron a **velocidad segura** (menor o igual a 120 km/h) y cuántas lo hicieron a **velocidad de riesgo** (mayor a 120 km/h). Al finalizar la lectura, se debe imprimir ambos contadores, además del **promedio** de todas las velocidades registradas.

Hint: Utiliza dos variables contador (una para cada categoría) y una variable acumuladora para la suma de velocidades, junto con otra que cuente el total de naves para calcular el promedio al final. No olvides evitar la división por cero si no se ingresó ninguna velocidad válida.

4. Suma de Dígitos y Validación

Escriba un programa en C que solicite al usuario un número entero positivo y calcule la **suma de sus dígitos**. Por ejemplo, si se ingresa 4831, la suma de sus dígitos es $4 + 8 + 3 + 1 = 16$.

Además, el programa debe determinar si dicha suma es un número **par** o **impar**, e imprimir el resultado correspondiente.

Ejercicio extra: Implemente esta lógica dentro de una **función** `int sumarDigitos(int numero)` que reciba el número y retorne la suma de sus dígitos, la cual debe ser invocada desde la función `main`.

Hint: Al igual que en el ejercicio de invertir un número, puedes usar el operador módulo (`% 10`) dentro de un ciclo `while` para ir extrayendo cada dígito uno a uno, sumándolo a un acumulador, mientras reduces el número original con división entera (`/ 10`) hasta que este llegue a 0.

5. Suma de los Primeros N Términos de Fibonacci

La secuencia de Fibonacci se define como: $F(0) = 0$, $F(1) = 1$, y $F(n) = F(n - 1) + F(n - 2)$ para $n \geq 2$. Escriba un programa en C que solicite al usuario un valor entero N (mayor o igual a 1) y calcule la **suma de los primeros N términos** de la secuencia de Fibonacci (comenzando desde $F(0)$).

Ejercicio extra: Implemente una **función recursiva** `int fibonacci(int n)` que calcule y retorne el n -ésimo término de la secuencia, y utilícela desde el `main` dentro de un ciclo para ir sumando cada uno de los N primeros términos.

Hint: La función recursiva debe considerar dos **casos base**: si $n = 0$ retorna 0, y si $n = 1$ retorna 1. Para cualquier otro caso, debe retornar la suma de `fibonacci(n-1)` y `fibonacci(n-2)`. Ten en cuenta que esta solución, aunque simple de escribir, es poco eficiente para valores grandes de N , ya que recalcula muchos términos repetidamente.